

Architettura degli Elaboratori

Architettura della CPU RISC-V



SAPIENZA
UNIVERSITÀ DI ROMA

Davide Polese

davide.polese@uniroma1.it

Special thanks and credits:

Alessandro Checco, Andrea Sterbini, Iacopo

Masi, Claudio di Ciccio

[S&PdC]

4.1-4.4



Informazioni sull'esonero

- Quando: 9 Maggio 2025 dalle 16 alle 19
- Dove: Aula 16 e 17 laboratori di via Tiburtina
- Prova pratica di assembly in rars
- Note:
 - necessaria la registrazione al link seguente
 - aperto solo a chi sta frequentando le lezioni

<https://forms.gle/vMMd9KzWXSx3UzCW9>

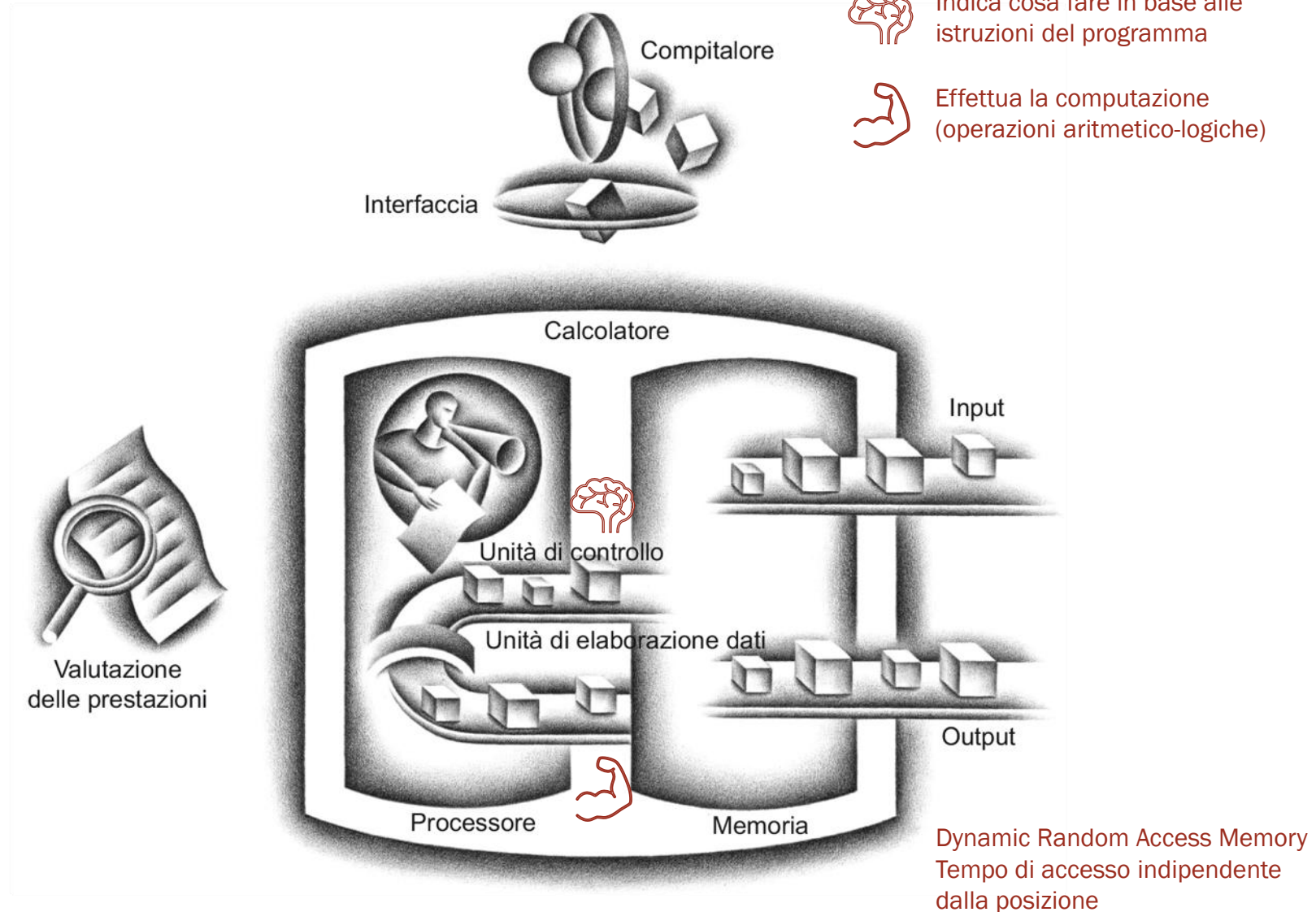
Argomenti

Argomenti della lezione

- Progetto della CPU RISC-V a singolo colpo di clock
 - Istruzioni da implementare
 - Unità funzionali necessarie
 - Datapath e unità di controllo
 - Tempo di esecuzione delle istruzioni



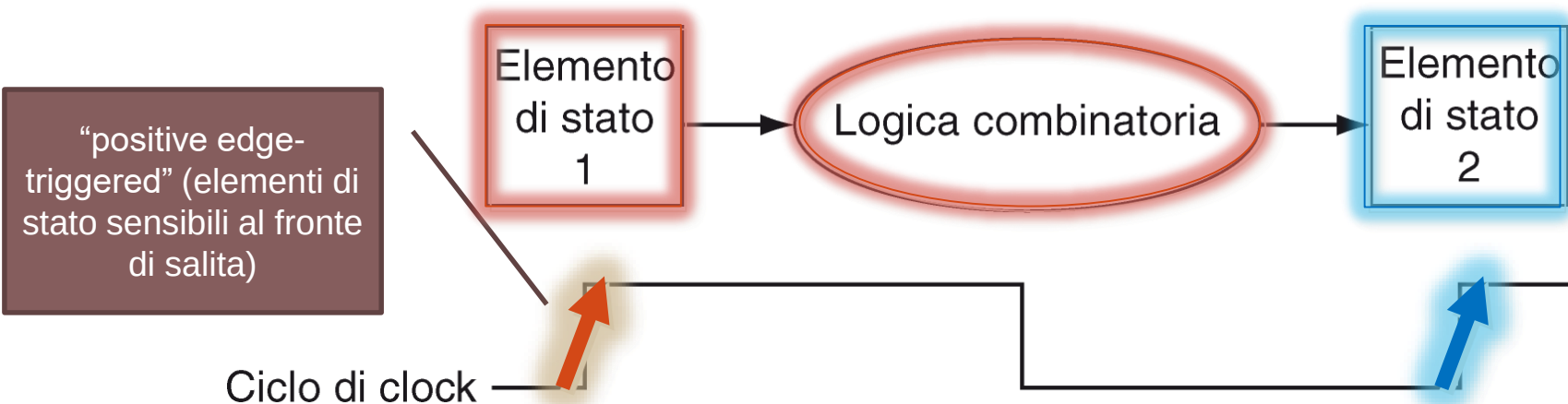
L'architettura del calcolatore



La CPU

CPU = macchina sequenziale ovvero Stato + circuito combinatorio

Rappresentazione numerica basata su valore di tensione (alto o basso → 1 o 0, tipicamente)

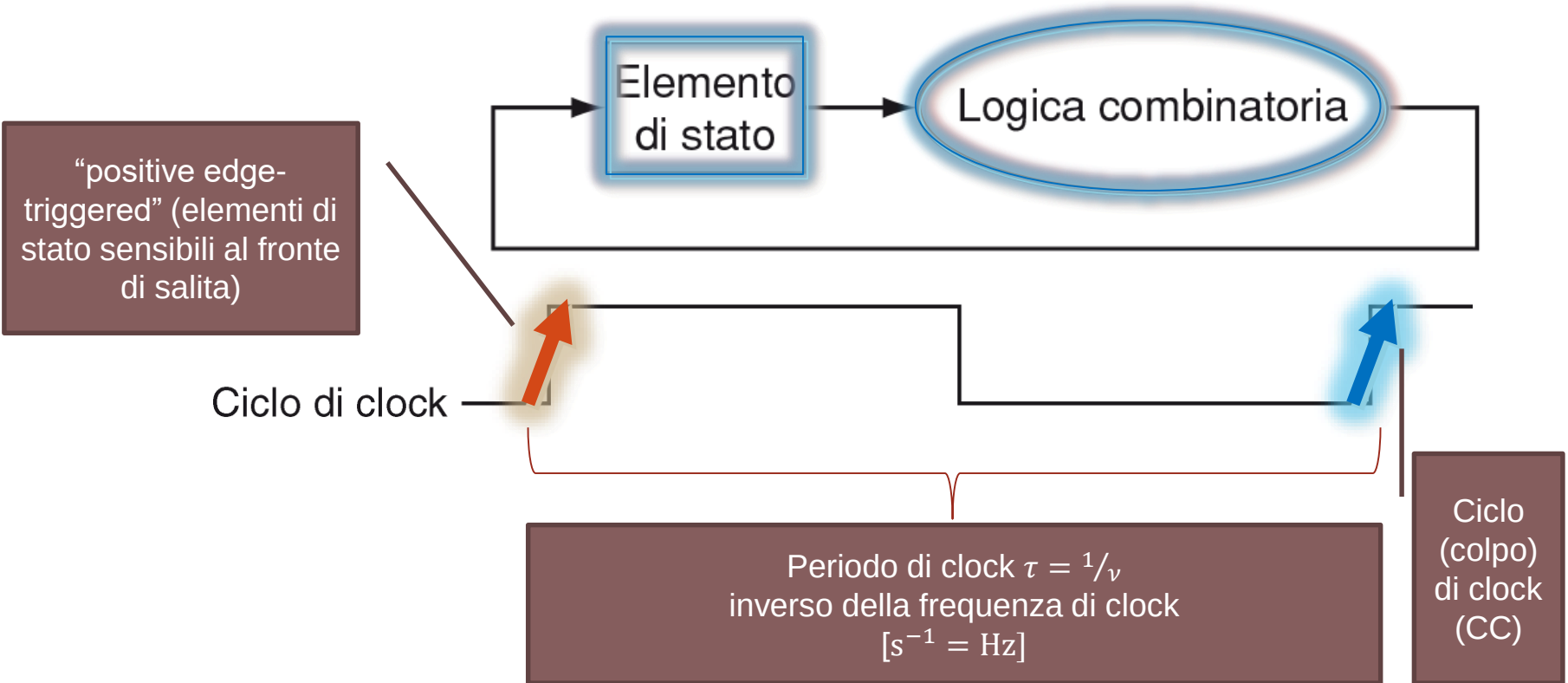


Tempo di esecuzione delle istruzioni ← durata dell'istruzione più lenta

La CPU

CPU = macchina sequenziale ovvero Stato + circuito combinatorio

Rappresentazione numerica basata su valore di tensione (alto o basso → 1 o 0, tipicamente)



Periodo di clock ← durata dell'istruzione più lenta

Progettare la CPU RISC-V

Prima fase: CPU RISC-V semplice non ottimizzata (senza pipeline)

- Definire come viene elaborata una istruzione (**fasi di esecuzione**)
- Scegliere le **istruzioni da realizzare**
- Scegliere le **unità funzionali necessarie**
- **Collegare** le unità funzionali
- **Costruire la CU (Control Unit)** che controlla il funzionamento della CPU
- Calcolare il massimo tempo di esecuzione delle istruzioni (che ci dà il **periodo di clock**)

Fasi di esecuzione di una istruzione:

Fetch	caricamento di una istruzione dalla memoria alla CU
Decodifica	decodifica della istruzione e lettura argomenti dai registri
Esecuzione	esecuzione (attivazione delle unità funzionali necessarie)
Memoria	accesso alla memoria
Write Back	scrittura dei risultati nei registri

Altre operazioni necessarie

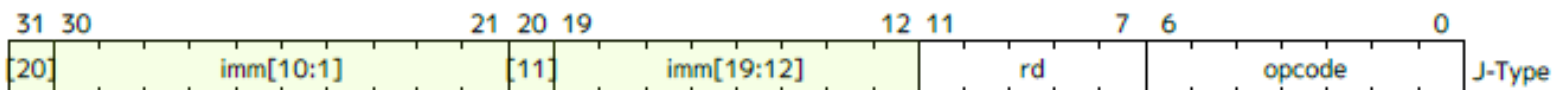
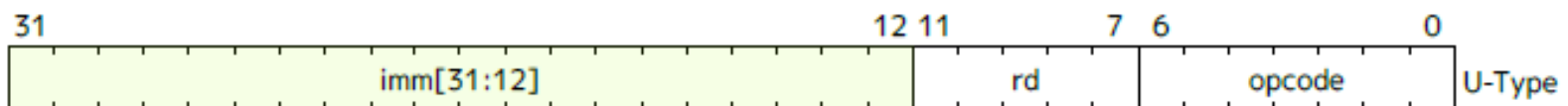
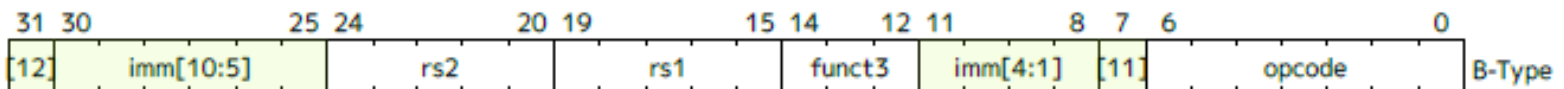
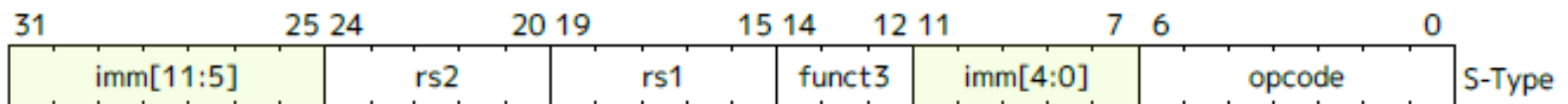
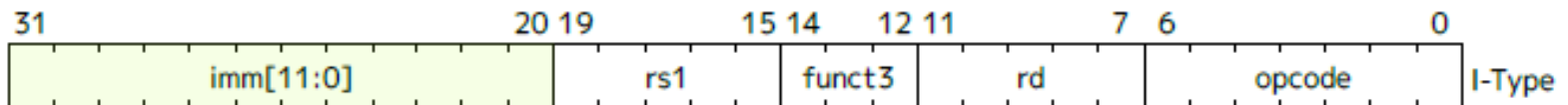
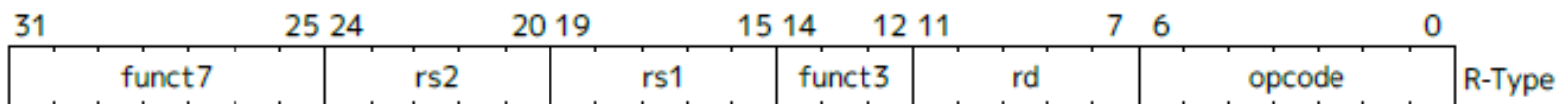
aggiornamento del PC (normale / salti condizionati / salti non condizionati)

Progettare la CPU (segue)

Istruzioni da realizzare

accesso alla memoria:	lw, sw	(di tipo I ed S)
salti condizionati:	beq	(di tipo SB)
operazioni aritmetico-logiche:	add, sub, and, or,...	(di tipo R)

Formato delle istruzioni RISC-V (ovvero la loro codifica in memoria)

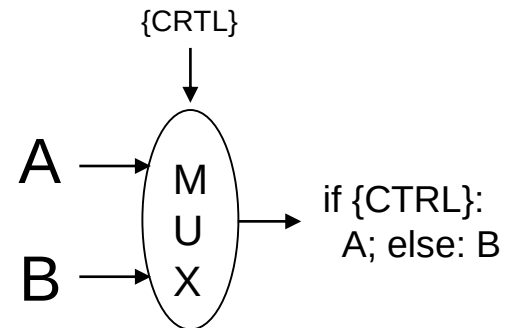


Unità funzionali necessarie

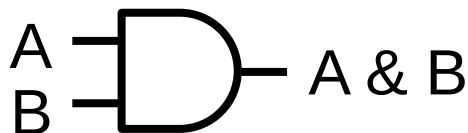
PC	registro che contiene l'indirizzo della istruzione
memoria istruzioni	contiene le istruzioni
adder	per calcolare il PC (successivo o salto)
registri	contengono gli argomenti delle istruzioni
ALU	fa le operazioni aritmetico-logiche, confronti, indirizzi in mem
memoria dati	da cui leggere/in cui scrivere i dati (load/store)

Unità collegate da diversi **datapath** (interconnessioni che definiscono il flusso delle informazioni nella CPU)

Se un'unità funzionale può ricevere dati da **più sorgenti** è necessario inserire un multiplexer (**MUX**) per selezionare la sorgente necessaria



Le unità funzionali sono attivate e coordinate dai segnali prodotti dalla **Control Unit**



Ingredienti: memoria delle istruzioni, PC, adder

Memoria istruzioni:

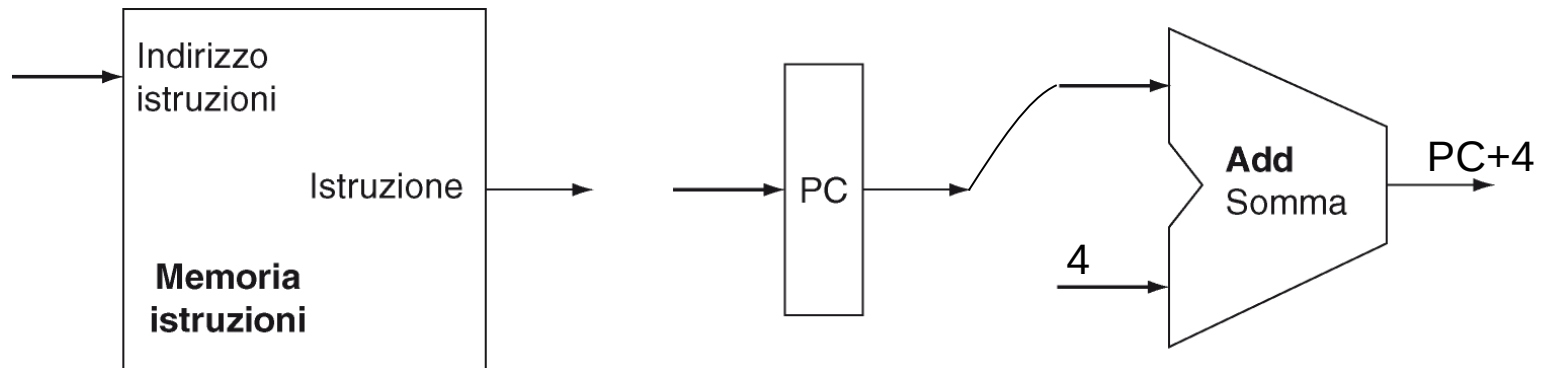
- Input: indirizzo a 32 bit
- Output: istruzione (da 32 bit) situata nell'indirizzo di input

Program Counter:

- Registro che contiene l'**indirizzo** della istruzione corrente

Sommatore:

- Necessario per calcolare il nuovo PC e le destinazioni dei salti relativi
- Riceve due valori a 32 bit e ne fornisce in uscita la somma



a. Memoria istruzioni

b. Program counter (PC)

c. Sommatore

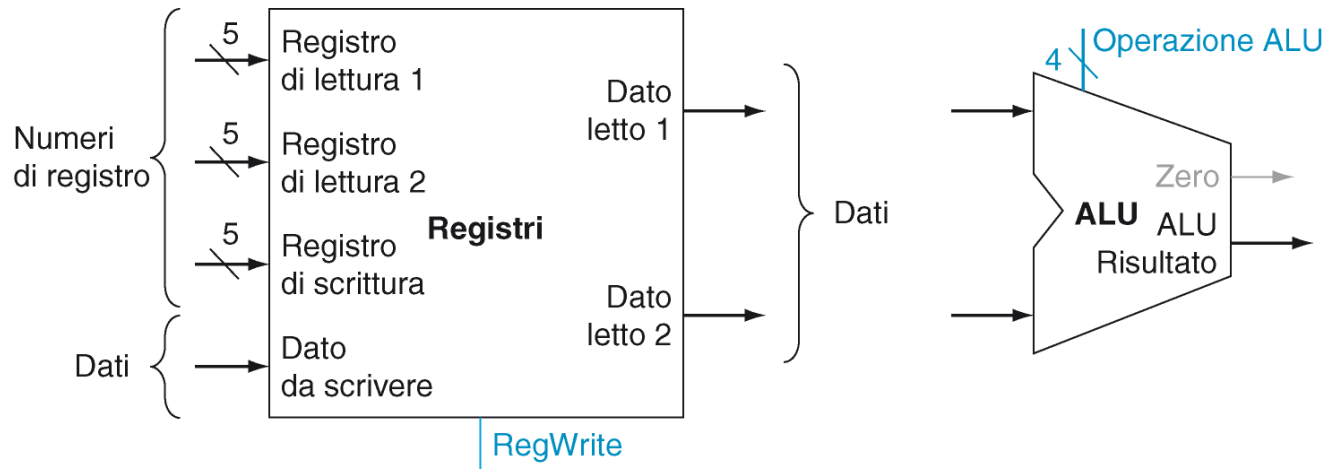
Ingredienti: registri e ALU

Blocco dei registri (register file):

- contiene **32 registri** a 32 bit, indirizzabili con 5 bit ($2^5 = 32$)
- può memorizzare un dato in un registro e contemporaneamente fornirlo in uscita
- 3 porte a 5 bit per indicare quali 2 registri leggere e quale registro scrivere
- 3 porte dati (a 32 bit)
 - una in ingresso per il valore da memorizzare e
 - 2 di uscita per i valori letti
- il segnale **RegWrite** abilita (se 1) la scrittura nel registro di scrittura

ALU: riceve due valori interi a 32 bit e svolge una operazione indicata dai segnali **Op. ALU**

- Oltre al risultato da 32 bit produce un segnale da 1 bit «**Zero**» asserito se il risultato è zero

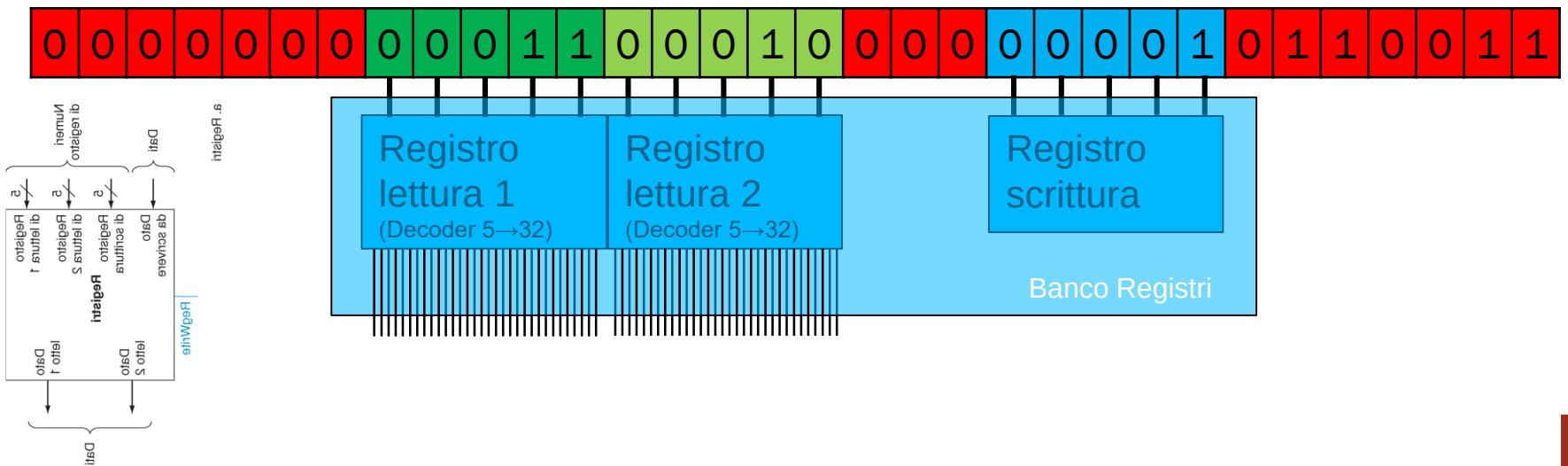
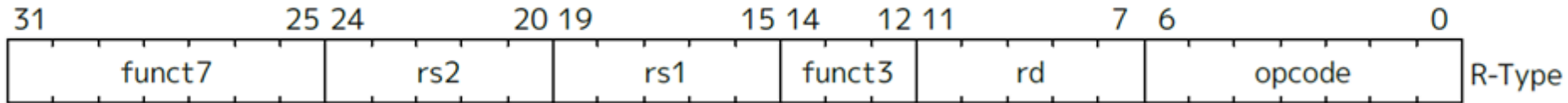


a. Registri

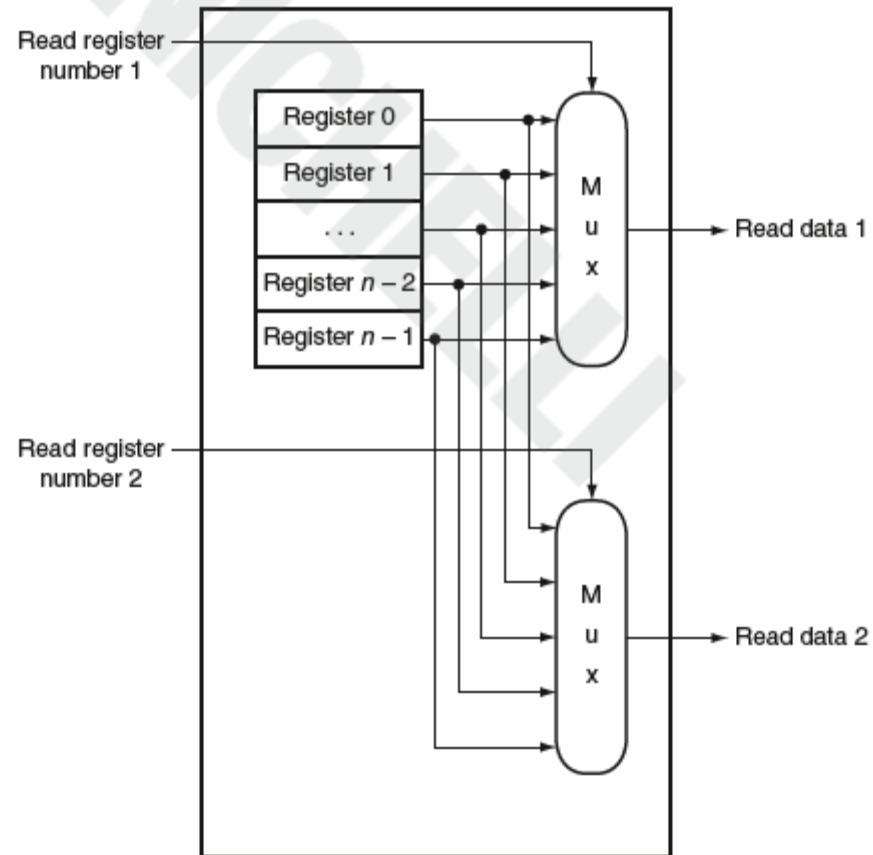
b. ALU

Istruzione – Accesso al banco registri

add x1, x2, x3



File register



File register

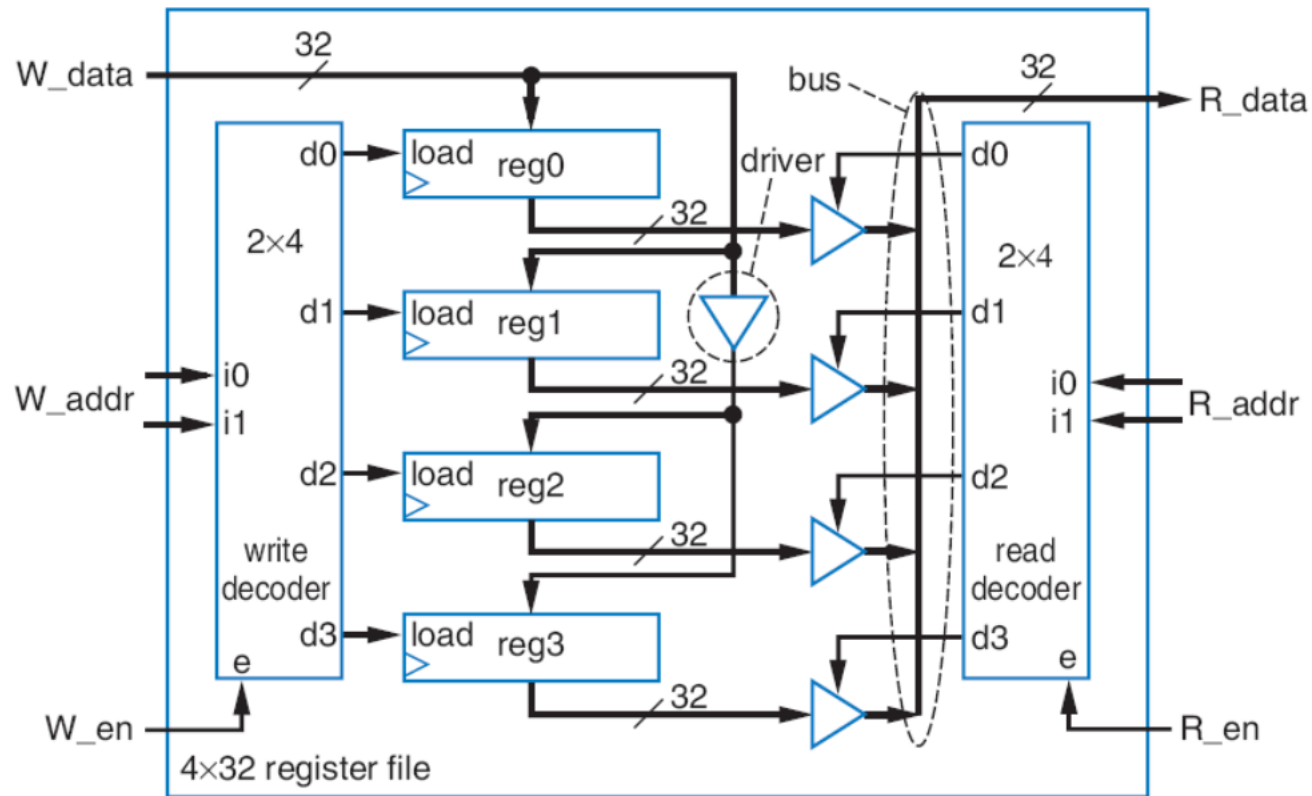


Figure L7-2. Implementation of a $2^2 \times 5$ Register File

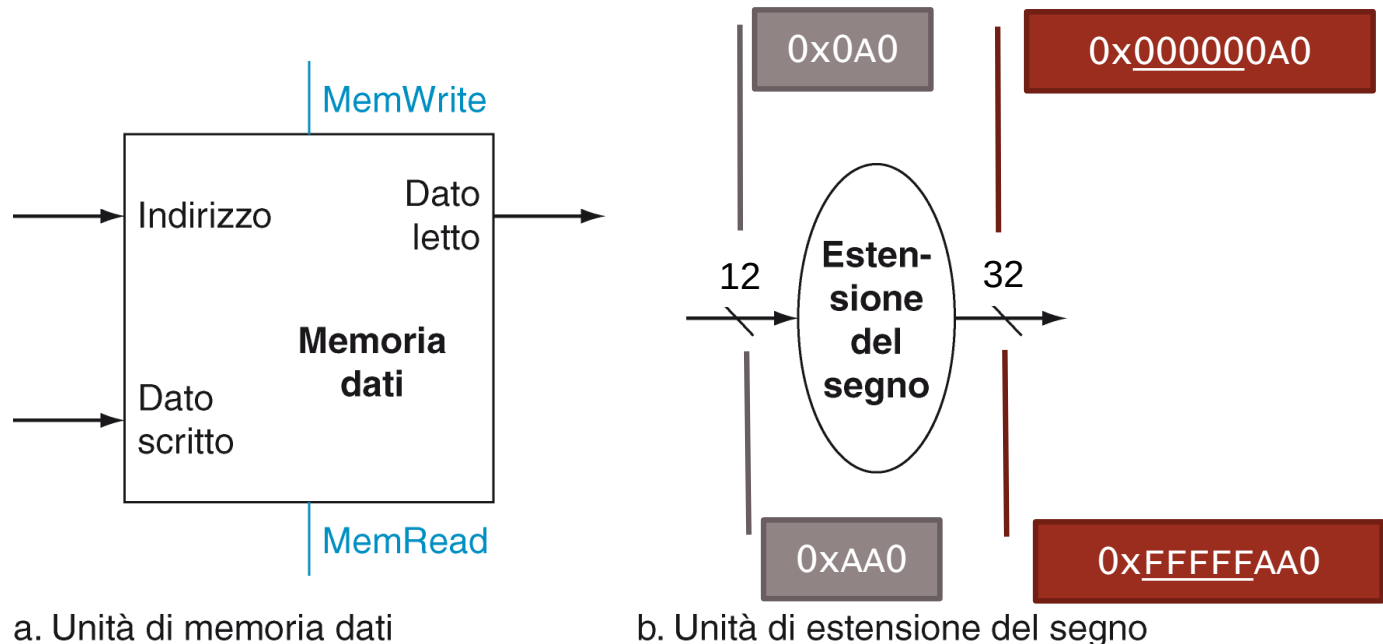
Ingredienti: memoria dati ed unità di estensione del segno

Unità di memoria

- riceve un **indirizzo** (da 32 bit) che indica quale word della memoria va letta/scritta
- riceve il segnale **MemRead** che abilita la lettura dall'indirizzo (se lw)
- riceve un dato da 32 bit da scrivere in memoria a quell'indirizzo (se sw)
- riceve il segnale di controllo **MemWrite** che abilita (1) la scrittura del dato all'indirizzo
- fornisce su una porta di uscita da 32 bit il lato letto (se **MemRead** = 1)

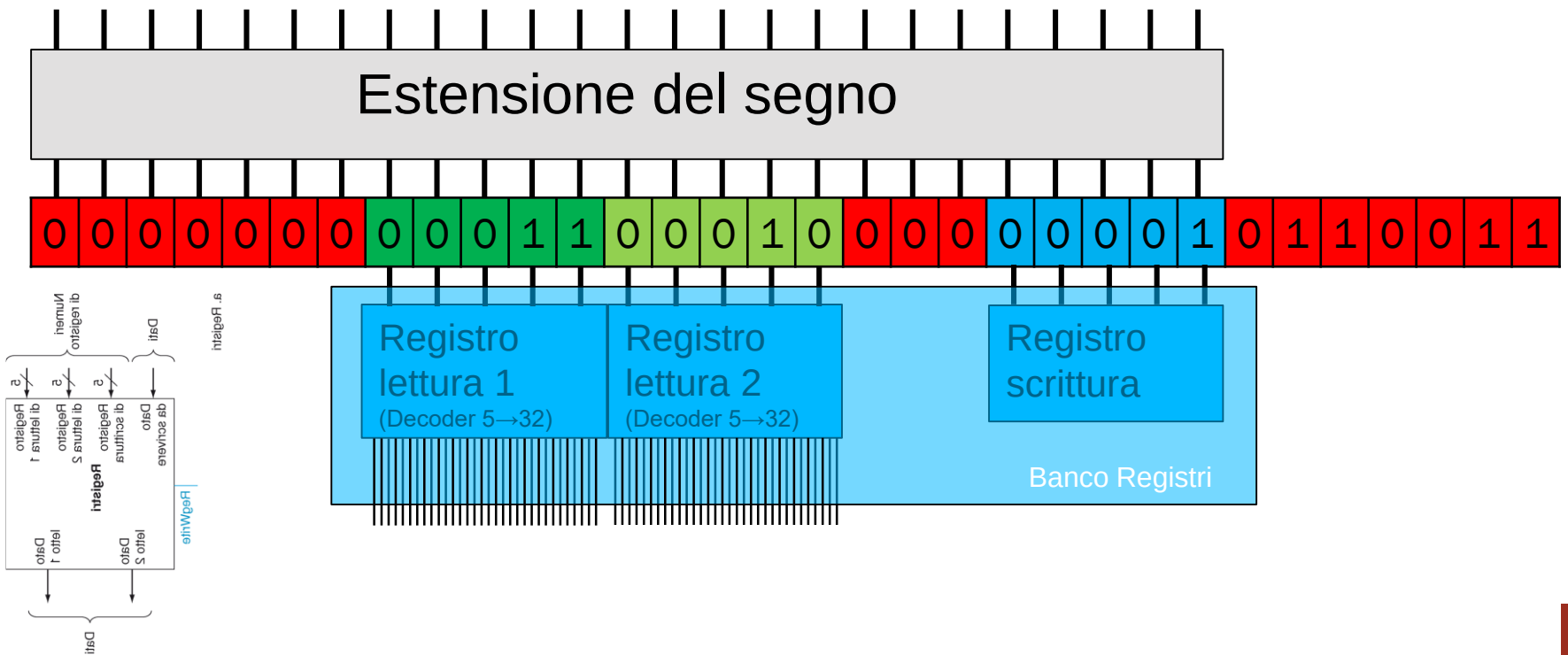
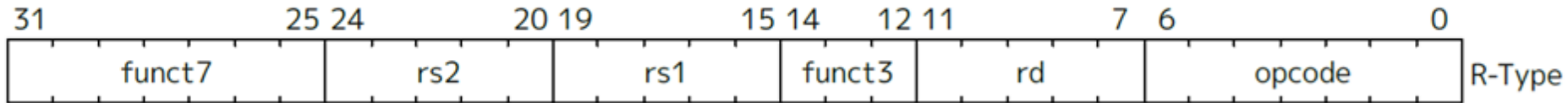
L'unità di estensione del segno serve a trasformare un intero relativo (in CA2) da 16 a 32 bit

- Ovvero copia il bit del segno nei 16 bit più significativi della parola



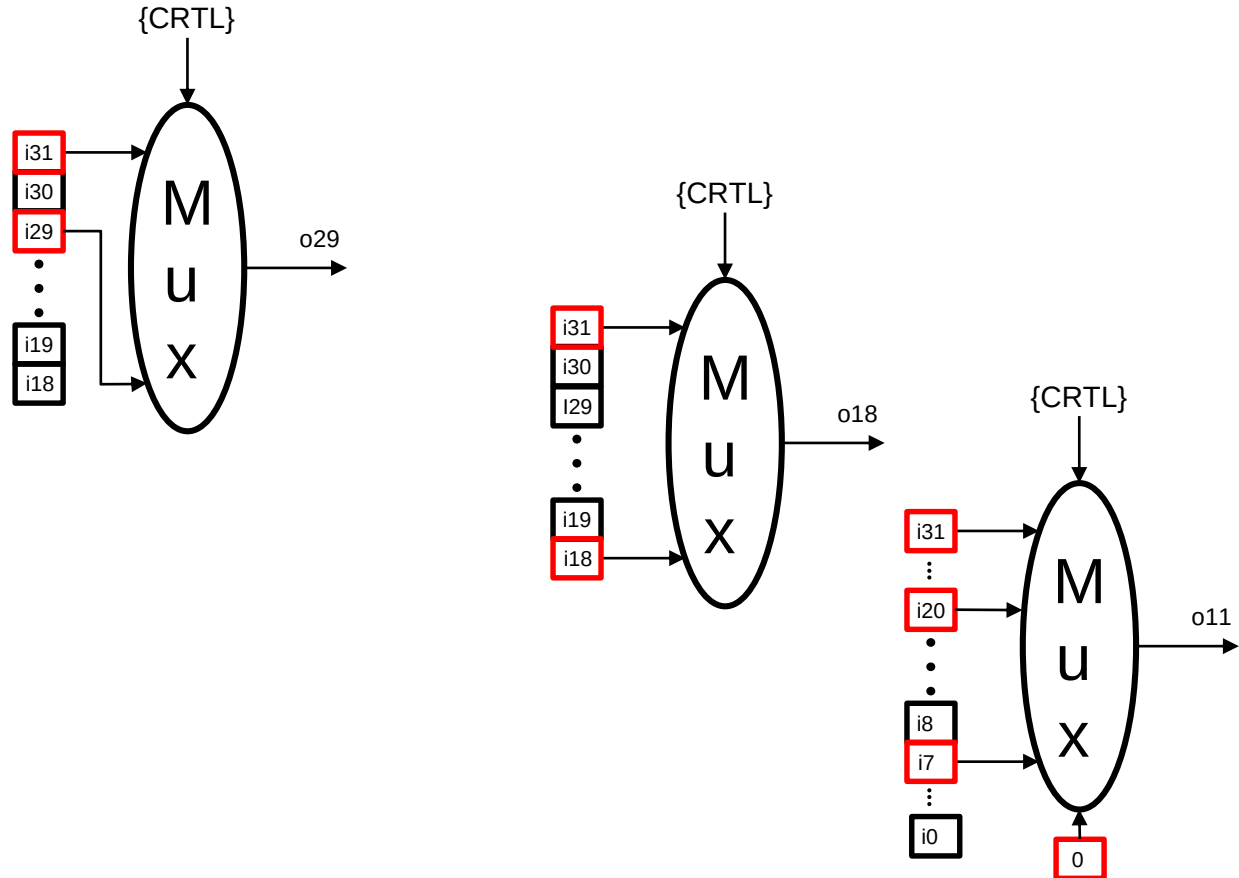
Istruzione – Accesso al banco registri + estensione del segno

add x1, x2, x3



Estensione del segno

		Campo immediato generato bit per bit																																
		31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Istruzione	Formato	Input del campo immediato bit per bit																																
Load, Aritm imm	I	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i30	i29	i28	i27	i26	i25	i24	i23	i22	i21	i20	
Store	S	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	i11	i10	i9	i8	i7
Salto cond	SB	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	i7	“	“	“	“	“	“	“	“	“	“	0
Salto incond	UJ	“	“	“	“	“	“	“	“	“	“	“	“	“	i19	i18	i17	i16	i15	i14	i13	i12	i20	“	“	“	“	“	“	“	“	“	“	
Load Upper imm	U	“	i30	i29	i28	i27	i26	i25	i24	i23	i22	i21	i20	“	“	“	“	“	“	“	“	0	0	0	0	0	0	0	0	0	0	0	“	
Num input		1	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	4	2	2	2	2	2	2	3	3	3	3	3	

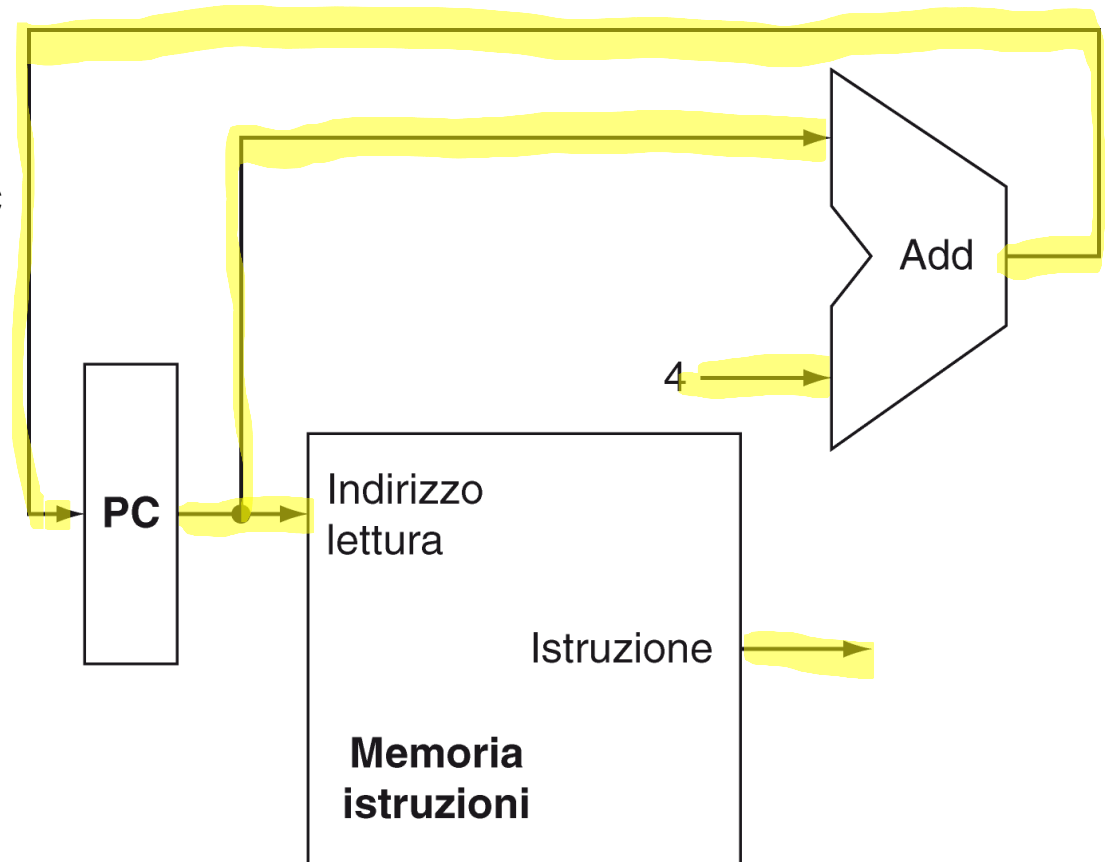


Fetch della istruzione/aggiornamento PC

- PC = indirizzo dell'istruzione
- Lettura dell'istruzione
- PC incrementato di 4 (1 word)
- Valore aggiornato reimpresso nel PC

NOTE

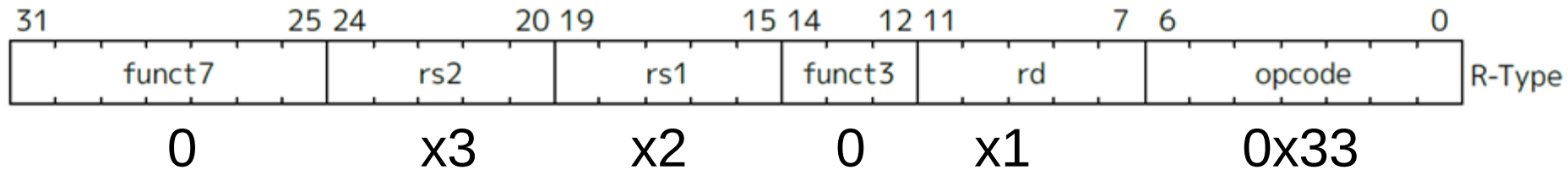
- Connessioni da 32 bit
- Mentre viene letta l'istruzione viene già calcolato il nuovo PC



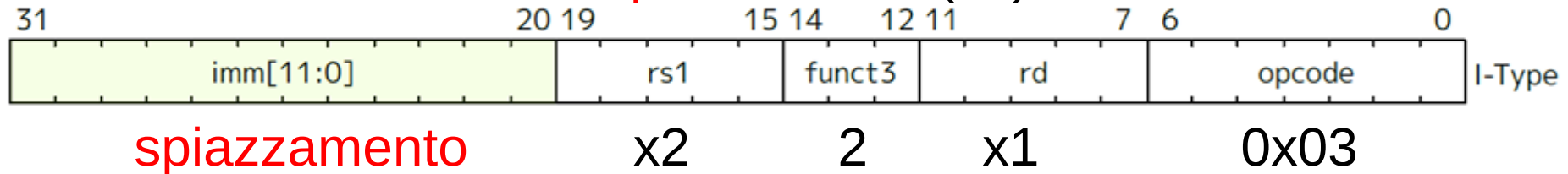
Dettaglio sulle istruzioni

Indice del registro destinazione in campi diversi dell'istruzione

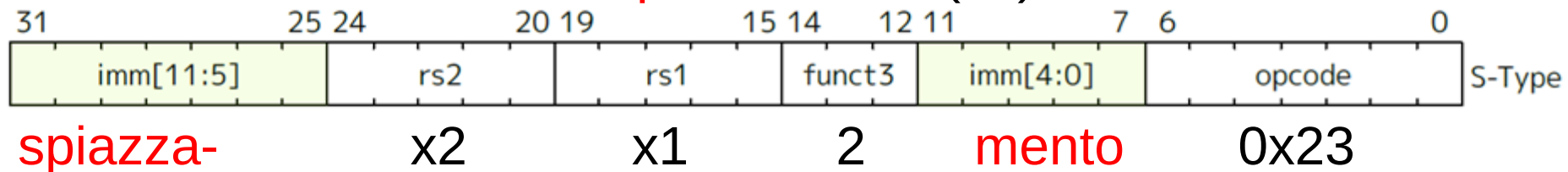
add x1, x2, x3



lw x1, spiazzamento(x2)



sw x1, spiazzamento(x2)



Tutte le istruzioni R hanno lo stesso opcode (0x33) e sono distinte dai campi funct3 e funct7

Operazioni ALU e accesso a MEM (ADD)

|funct7|rs2|rs1|funct3|rd|opcode|
|0|s2|s1|0|s0|0x33|

Decodifica facilitata: i formati I, R, e S sono simili

Primo argomento dell'istruzione: **registro** (rs1 in [19:15])

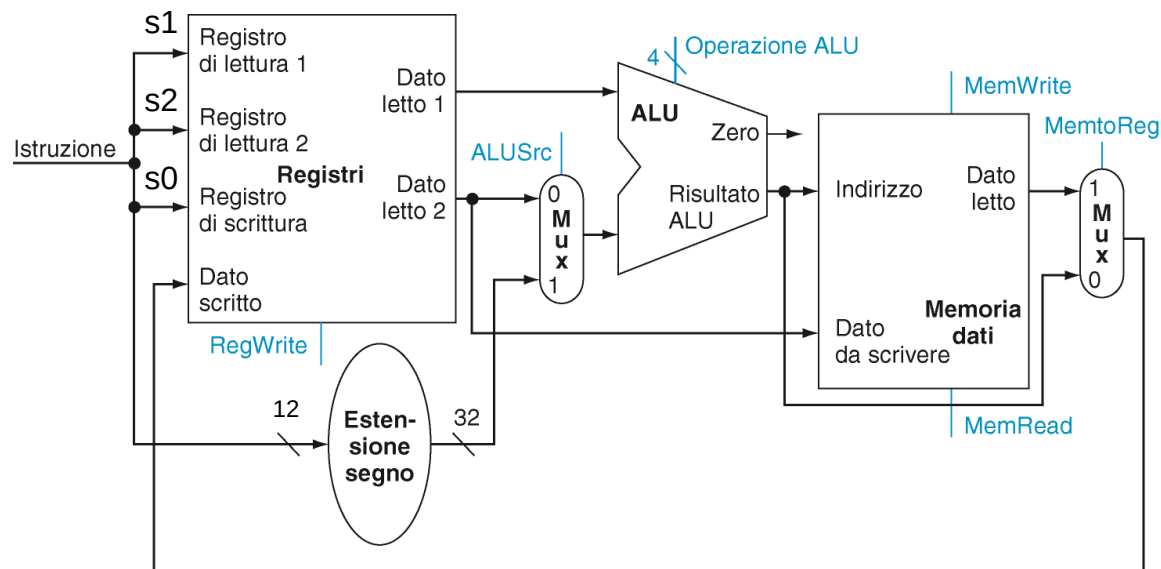
Secondo argomento dell'istruzione: **registro** (rs2 in [24:20]) o **campo immediato** ([31:20] per tipo I e [31:25],[11:7] per tipo S)

Registro di scrittura (rd in [11:7] per R e I, assente per tipo S che non scrive in un registro)

a seconda del segnale di controllo **ALUSrc** che seleziona la porta corrispondente del MUX

Per calcolare l'indirizzo di accesso alla memoria si usa la stessa ALU (reg. base + campo i.)

Il risultato dell'ALU o della memoria viene selezionato da **MementoReg** che comanda il MUX a destra



Operazioni ALU e accesso a MEM (LW)

|imm|rs1|funct3|rd|opcode|
|4|s1|2|s0|3

Decodifica facilitata: i formati I, R, e S sono simili

Primo argomento dell'istruzione: **registro** (rs1 in [19:15])

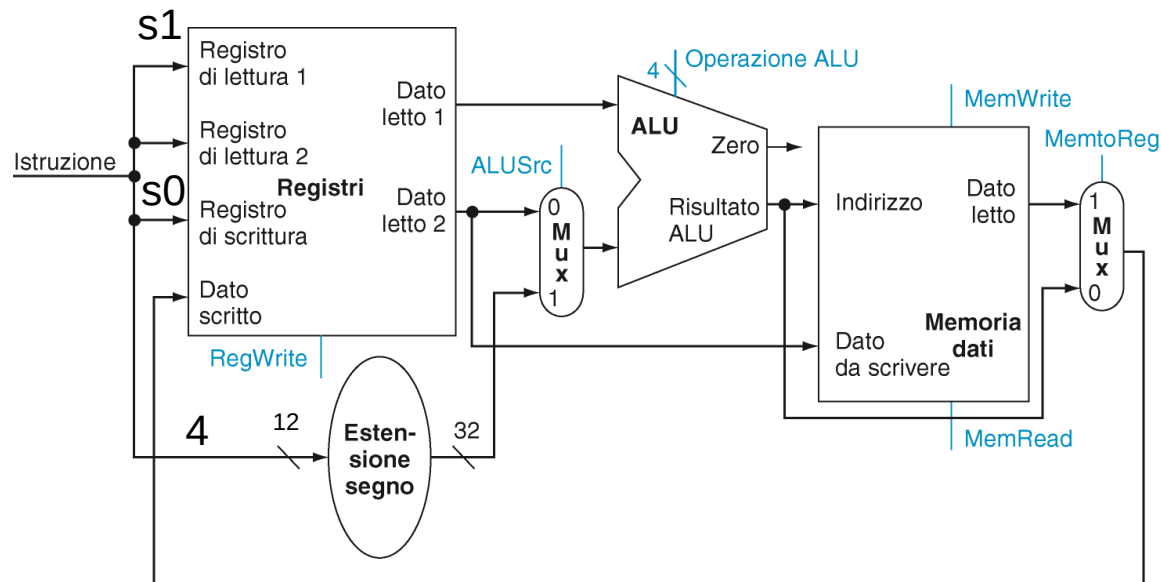
Secondo argomento dell'istruzione: **registro** (rs2 in [24:20]) o **campo immediato** ([31:20] per tipo I e [31:25],[11:7] per tipo S)

Registro di scrittura (rd in [11:7] per R e I, assente per tipo S che non scrive in un registro)

a seconda del segnale di controllo **ALUSrc** che seleziona la porta corrispondente del MUX

Per calcolare l'indirizzo di accesso alla memoria si usa la stessa ALU (reg. base + campo i.)

Il risultato dell'ALU o della memoria viene selezionato da **MementoReg** che comanda il MUX a destra



Operazioni ALU e accesso a MEM (SW)

|imm|rs2|rs1|funct3|imm|opcode|
|0|s0|s1|2|4|23|

Decodifica facilitata: i formati I, R, e S sono simili

Primo argomento dell'istruzione: **registro** (rs1 in [19:15])

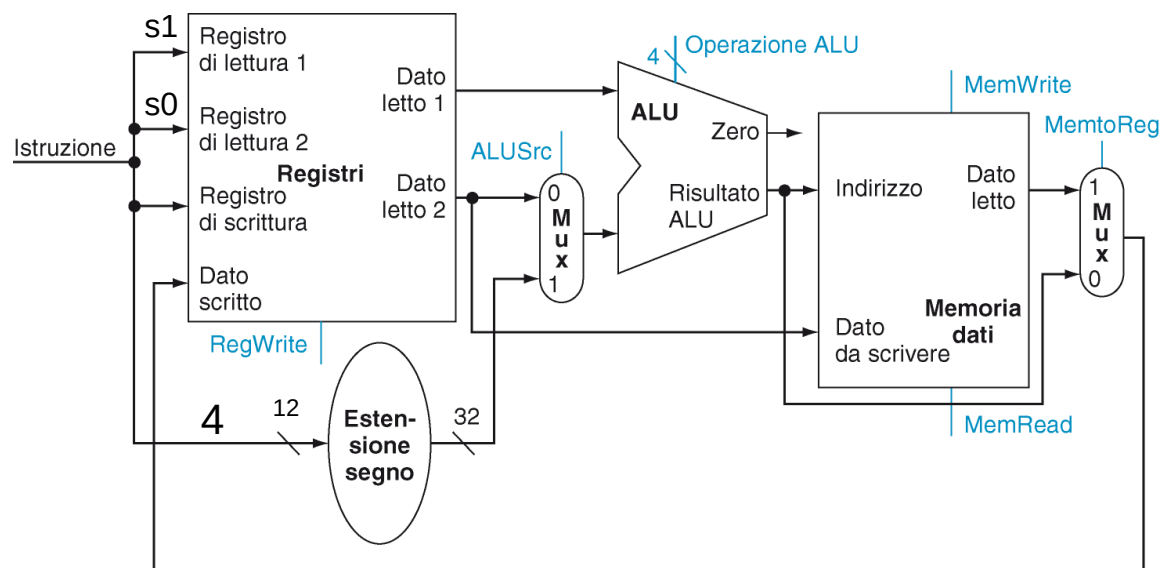
Secondo argomento dell'istruzione: **registro** (rs2 in [24:20]) o **campo immediato** ([31:20] per tipo I e [31:25],[11:7] per tipo S)

Registro di scrittura (rd in [11:7] per R e I, assente per tipo S che non scrive in un registro)

a seconda del segnale di controllo **ALUSrc** che seleziona la porta corrispondente del MUX

Per calcolare l'indirizzo di accesso alla memoria si usa la stessa ALU (reg. base + campo i.)

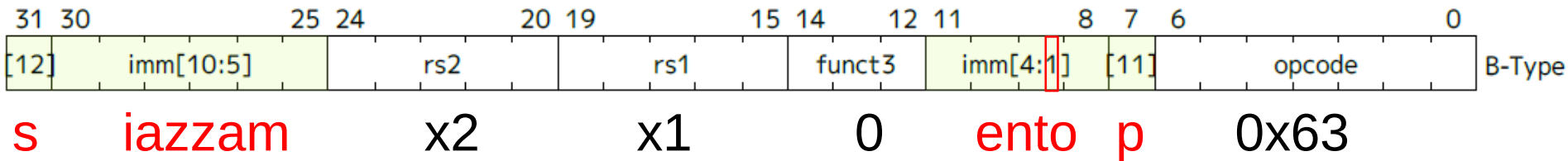
Il risultato dell'ALU o della memoria viene selezionato da **MementoReg** che comanda il MUX a destra



Dettaglio sulle istruzioni (BEQ)

Indice del registro destinazione in campi diversi dell'istruzione

beq s1, s2, **spiazzamento**



Lo spiazzamento è in mezza parole

Salti condizionati (beq)

ALU come **comparatore** (sottrazione)

- Il segnale **Zero** indica se operare il salto

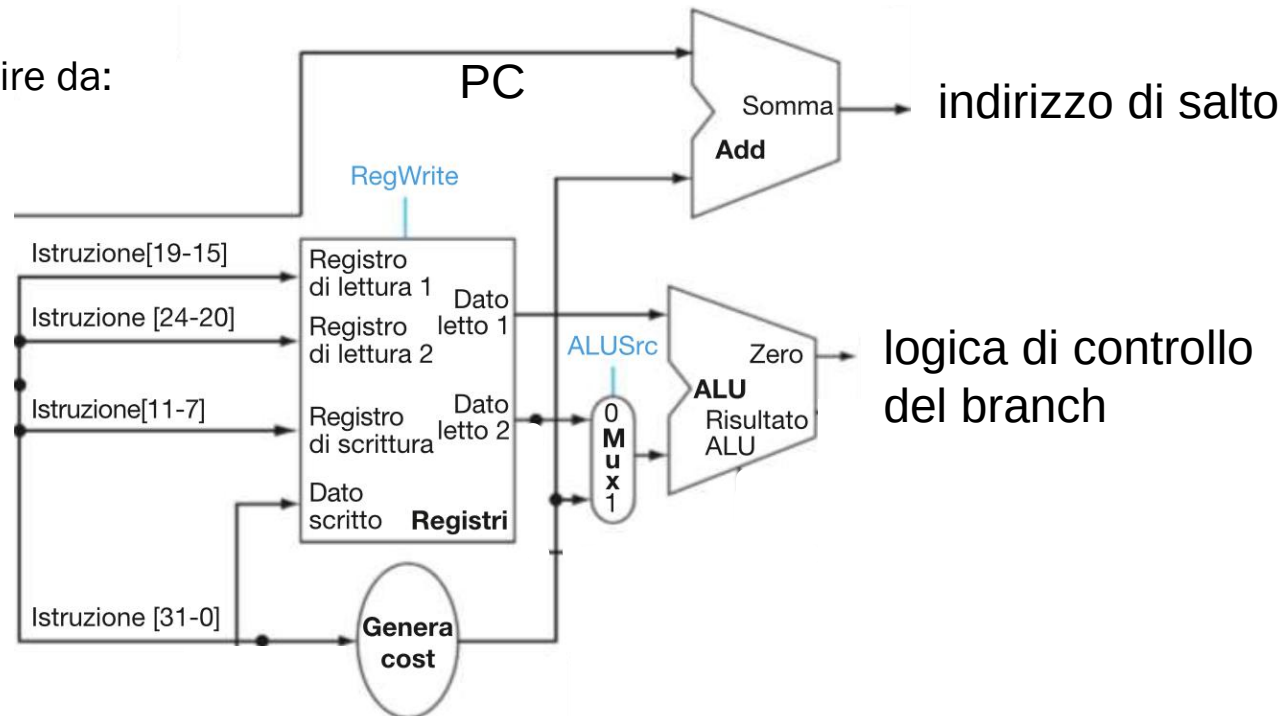
La destinazione dei salti è un **numero relativo di istruzioni** rispetto al PC:

- estesa nel segno (Genera cost)
- moltiplicata per 2 (Genera cost)
- sommata a PC

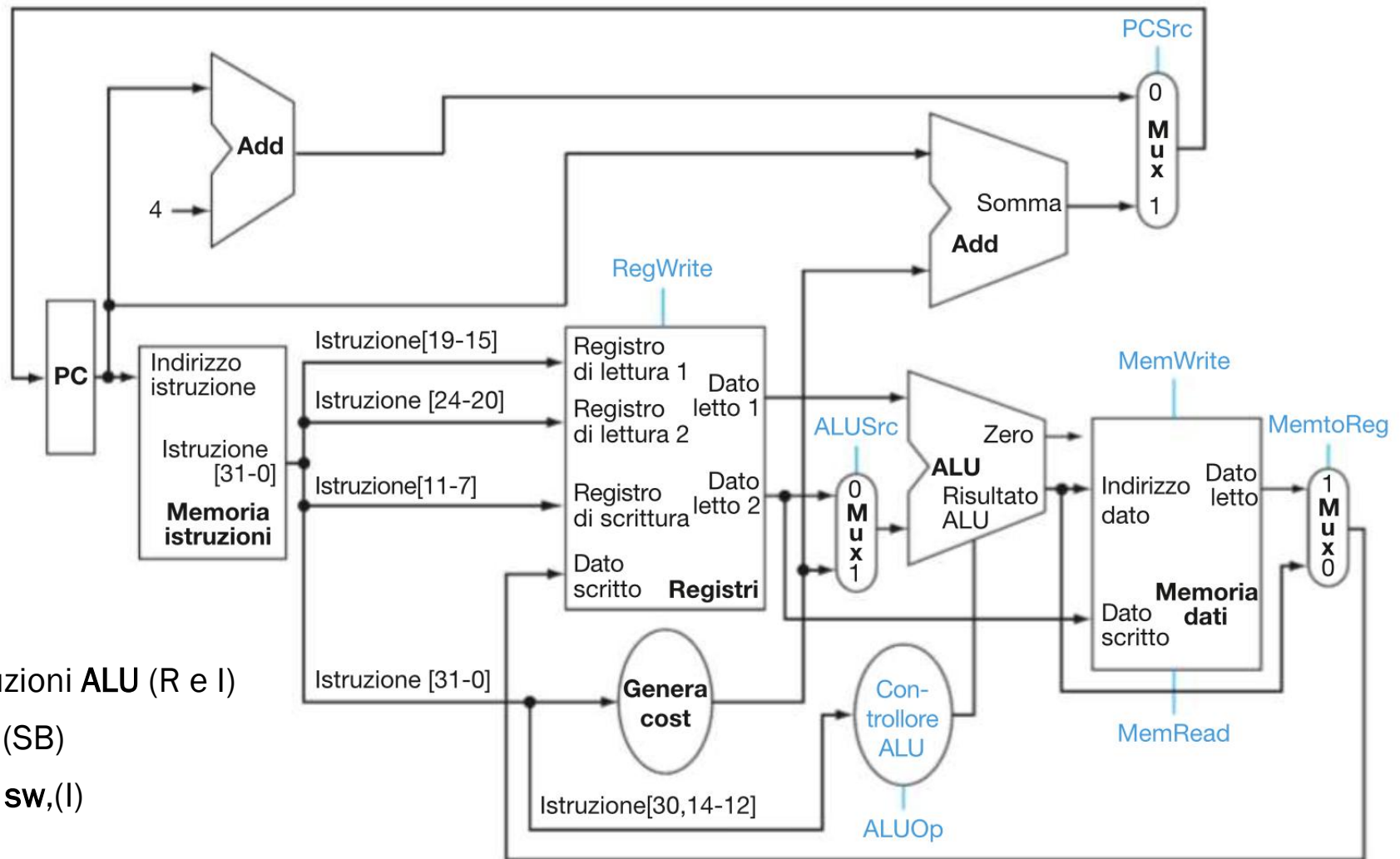
Il nuovo valore del PC può provenire da:

- PC+4 (istruzione seguente) o
- uscita dell'adder (salto)

Necessario un **MUX** di selezione

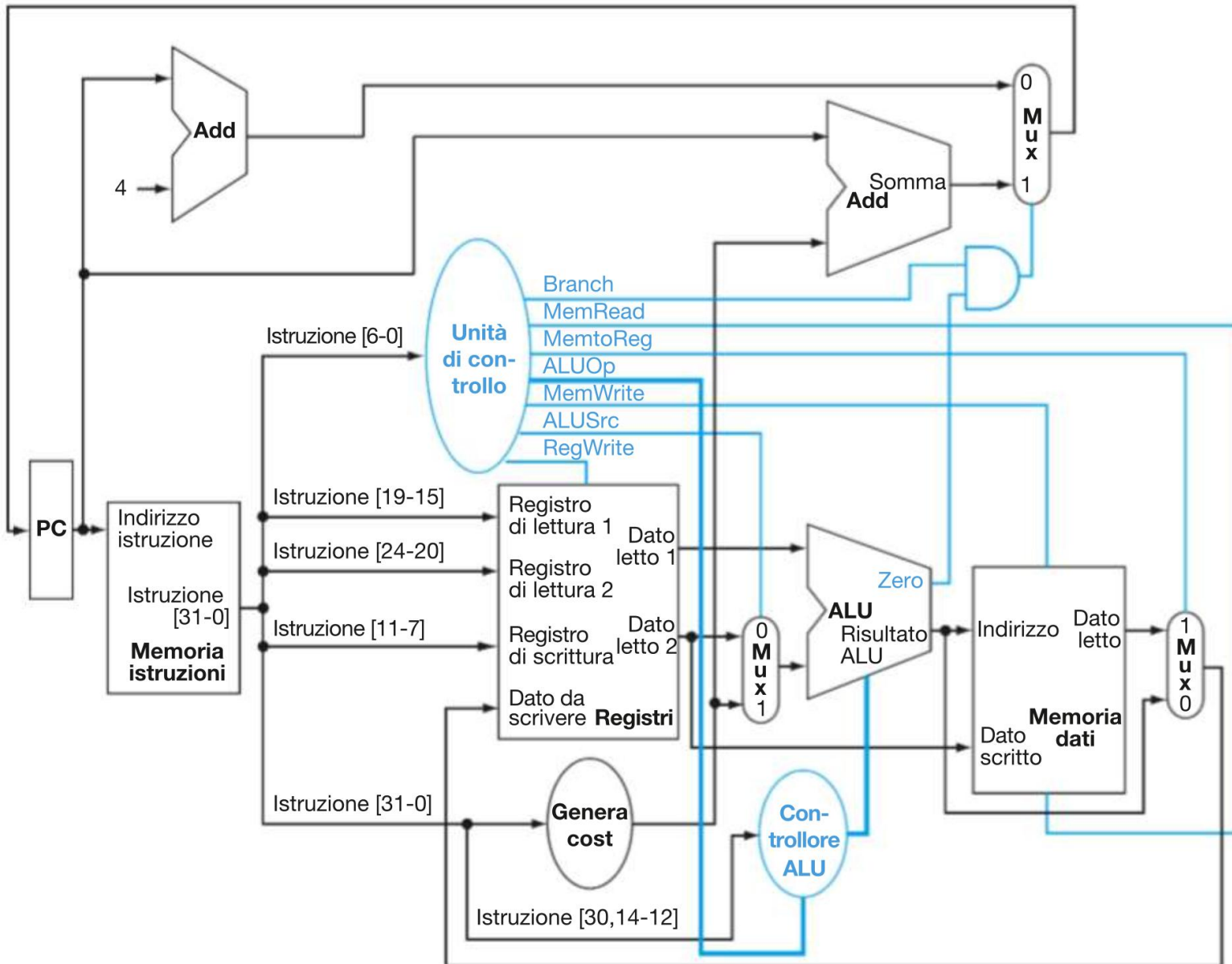


Tutto insieme

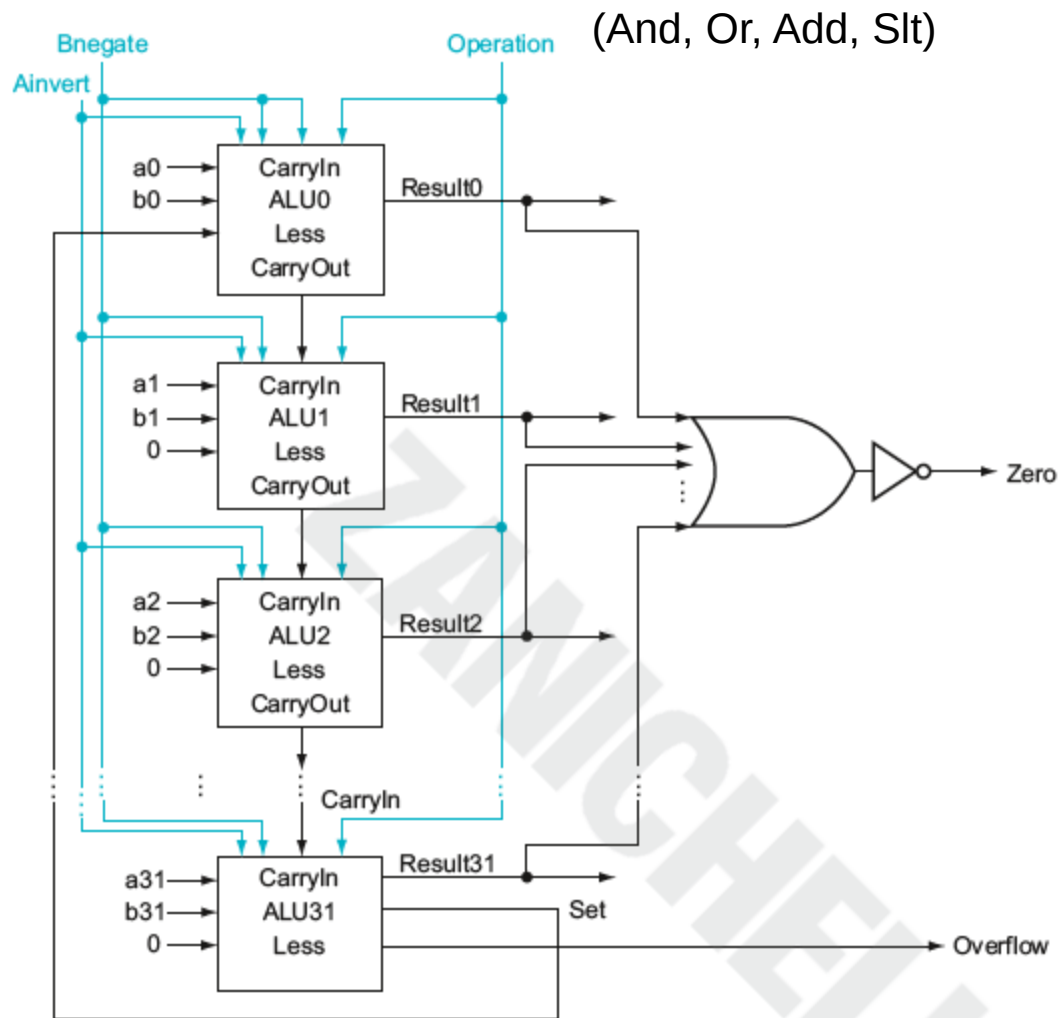


Istruzioni **ALU** (R e I)
beq (SB)
lw e **sw**, (I)

... con Control Unit e logica di branch



ALU



Unità di controllo della ALU

ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract

Formato istruzioni e bit di controllo ALU

La logica di controllo per la ALU è implementata a «cascata».

- Livelli multipli di decodifica.

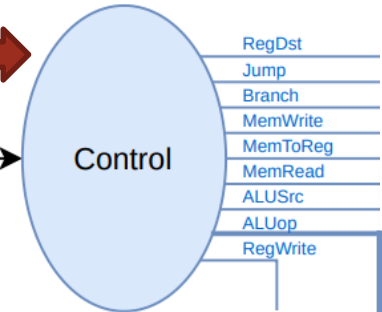
→ La circuiteria è più semplice.

Prima vi sono i 3 codici di selezione per ALUOp.

Poi, a seconda dei casi, viene considerato i campi **funz** nella istruzione.

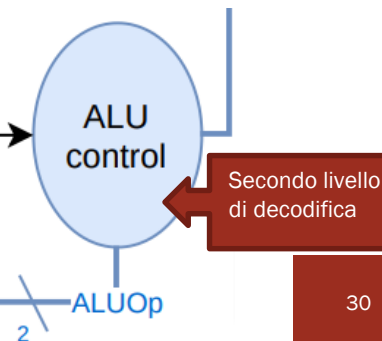
Primo livello di decodifica

Instr [6-0]



Codice operativo istruzione	ALUOp	Operazione eseguita dall'istruzione	Campo funz7	Campo funz3	Operazione dell'ALU	Ingresso di controllo alla ALU
lw	00	load di 1 parola	XXXXXXX	XXX	somma	0010
sw	00	store di 1 parola	XXXXXXX	XXX	somma	0010
beq	01	salto condizionato all'uguaglianza	XXXXXXX	XXX	sottrazione	0110
Tipo R	10	add	0000000	000	somma	0010
Tipo R	10	sub	0100000	000	sottrazione	0110
Tipo R	10	and	0000000	111	AND	0000
Tipo R	10	or	0000000	110	OR	0001

Instr [30,14-12]

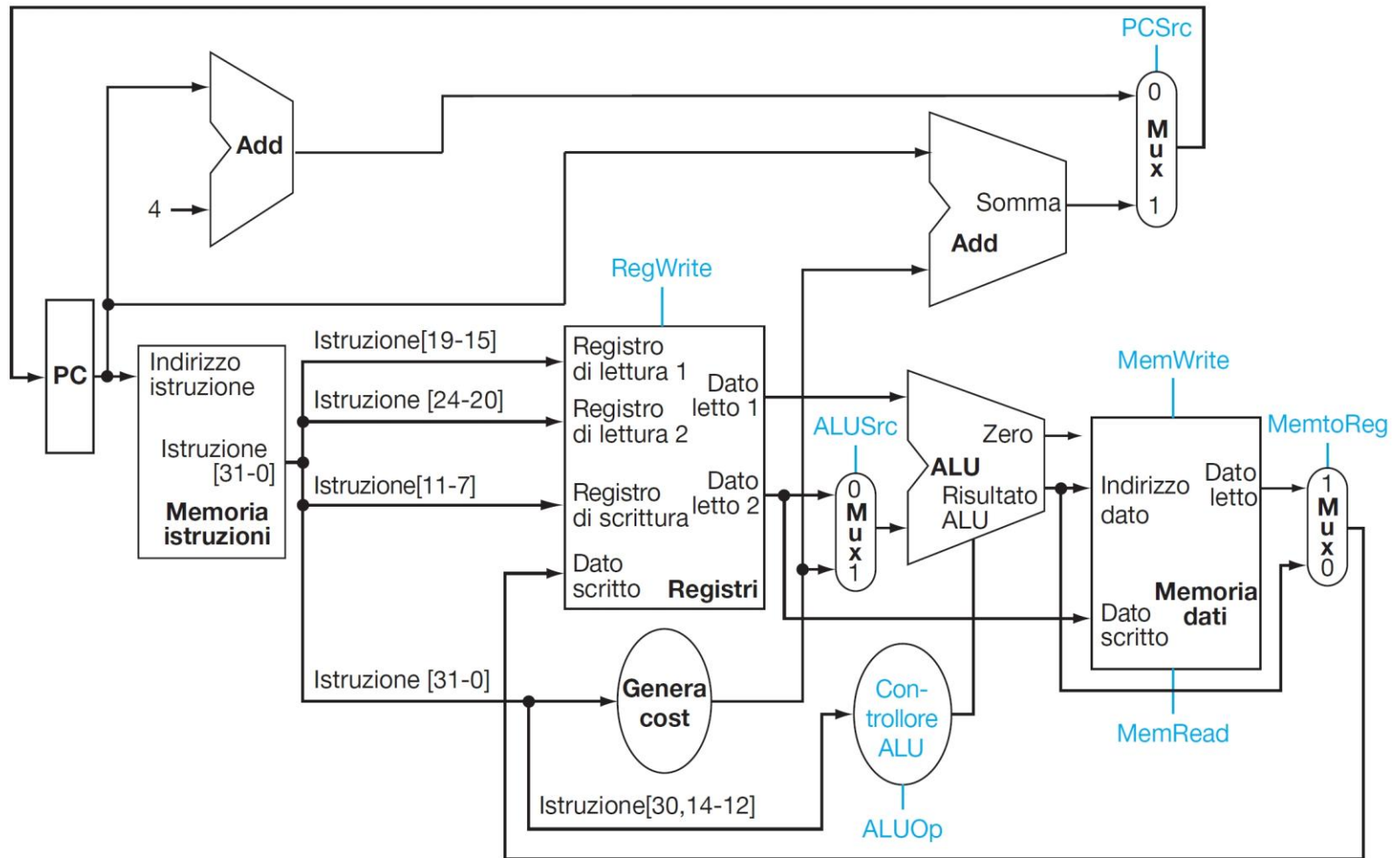


Secondo livello di decodifica

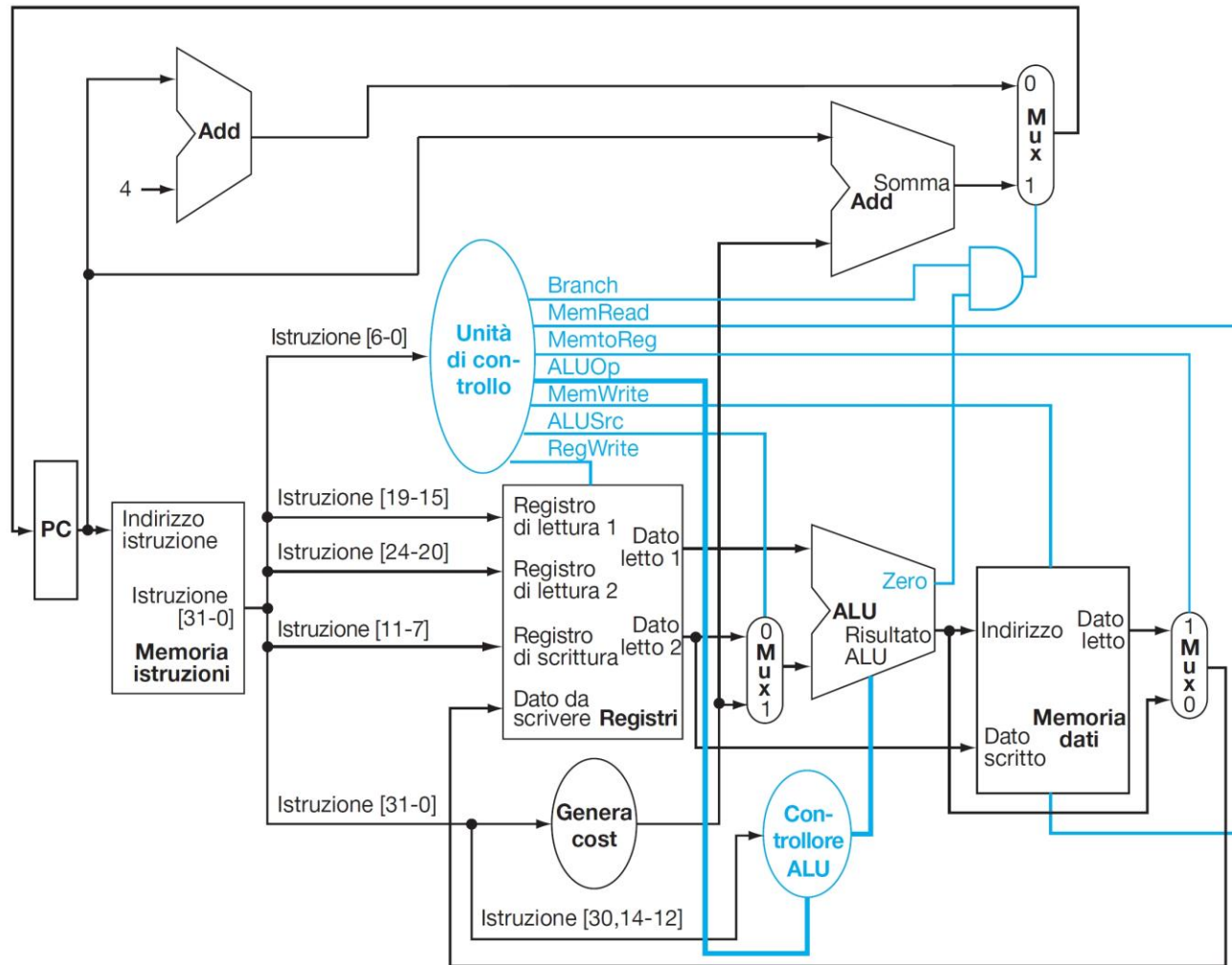
Input di controllo e tabella di verità

	ALUOp		Campo funz7							Campo funz3			Operazione
	ALUOp1	ALUOp2	I[31]	I[30]	I[29]	I[28]	I[27]	I[26]	I[25]	I[14]	I[13]	I[12]	
lw, sw	0	0	X	X	X	X	X	X	X	X	X	X	0010 
beq	X	1	X	X	X	X	X	X	X	X	X	X	0110 
add	1	X	0	0	0	0	0	0	0	0	0	0	0010 
sub	1	X	0	1	0	0	0	0	0	0	0	0	0110 
and	1	X	0	0	0	0	0	0	0	1	1	1	0000
or	1	X	0	0	0	0	0	0	0	1	1	0	0001

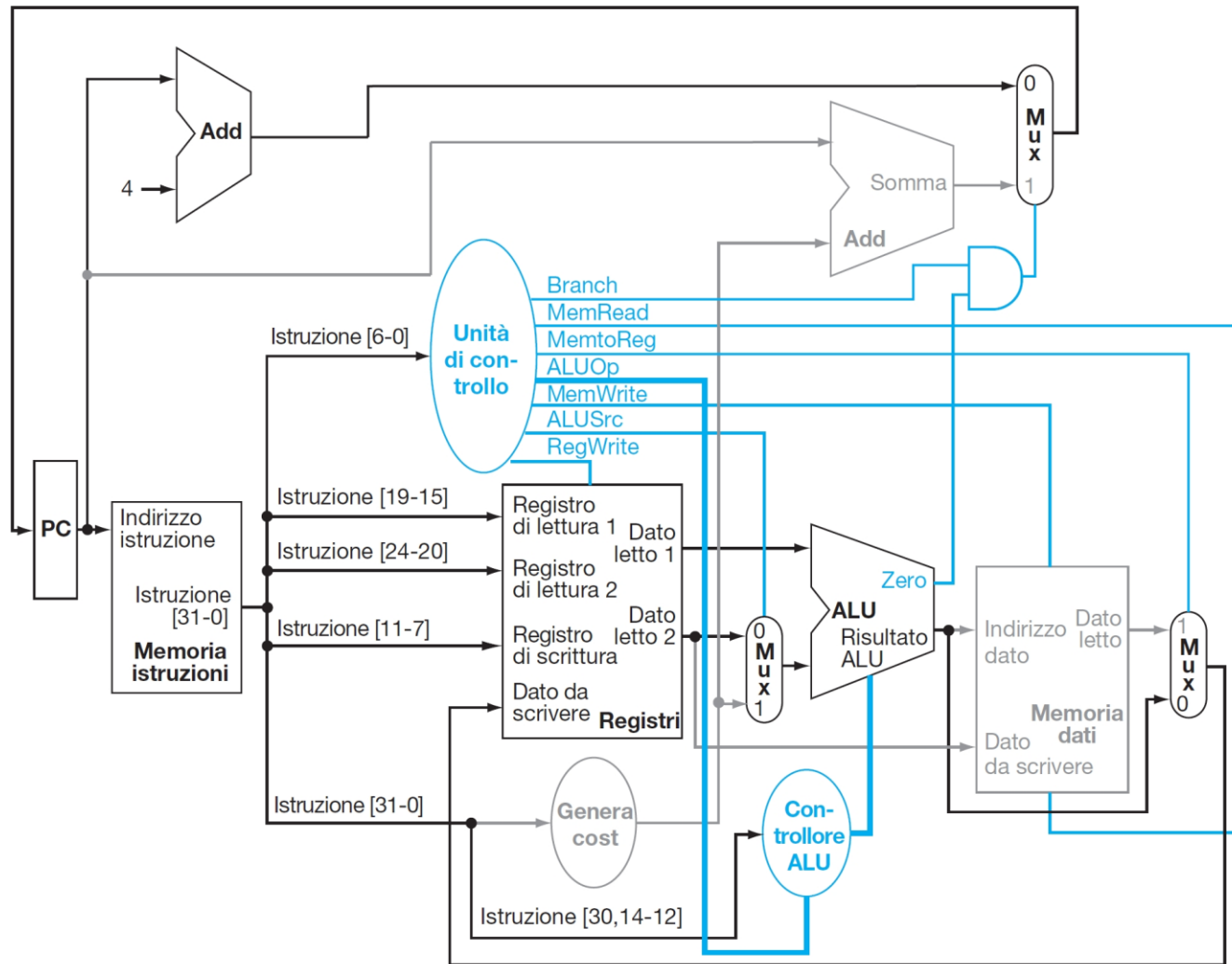
Datapath completo (senza CU)



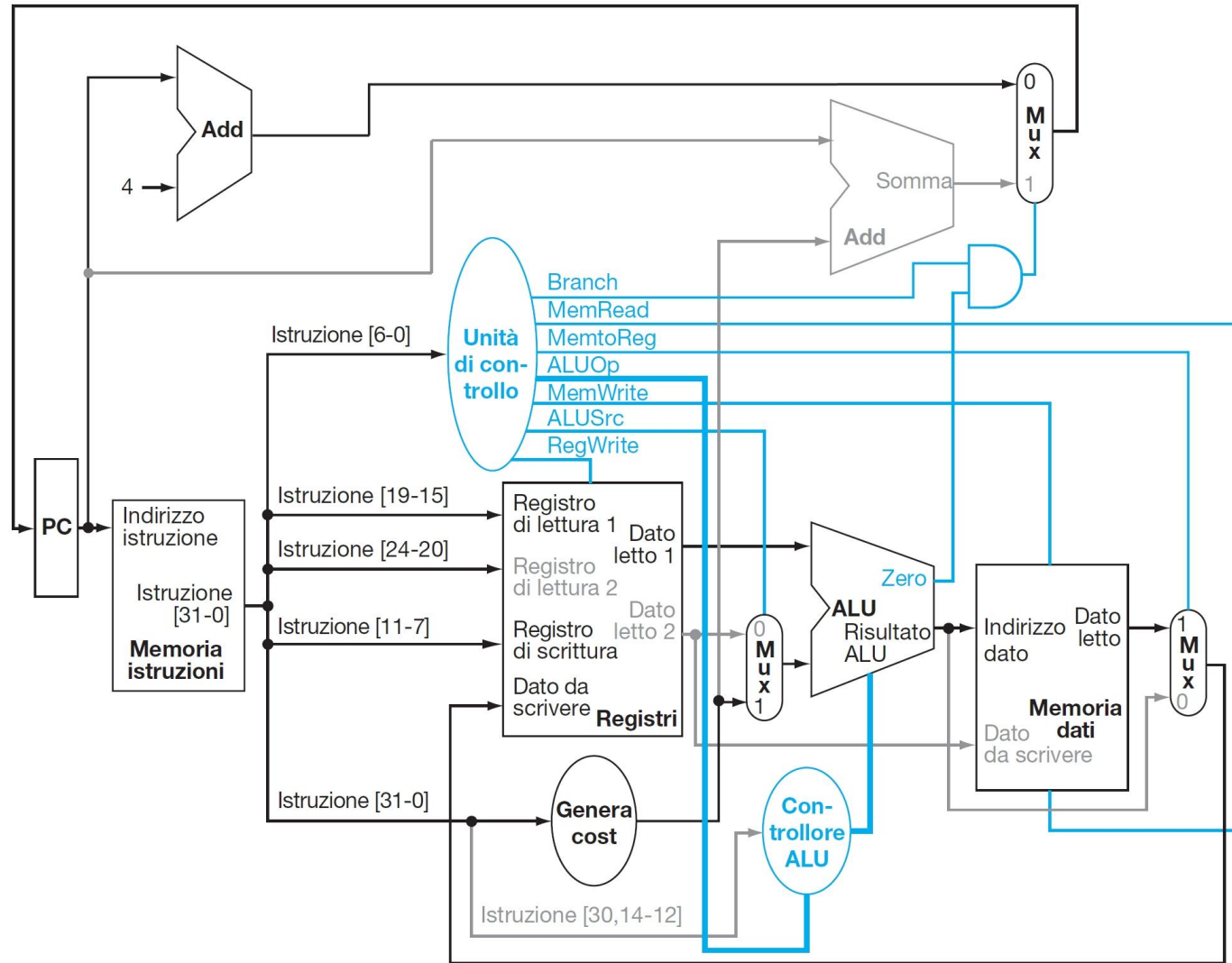
Datapath completo



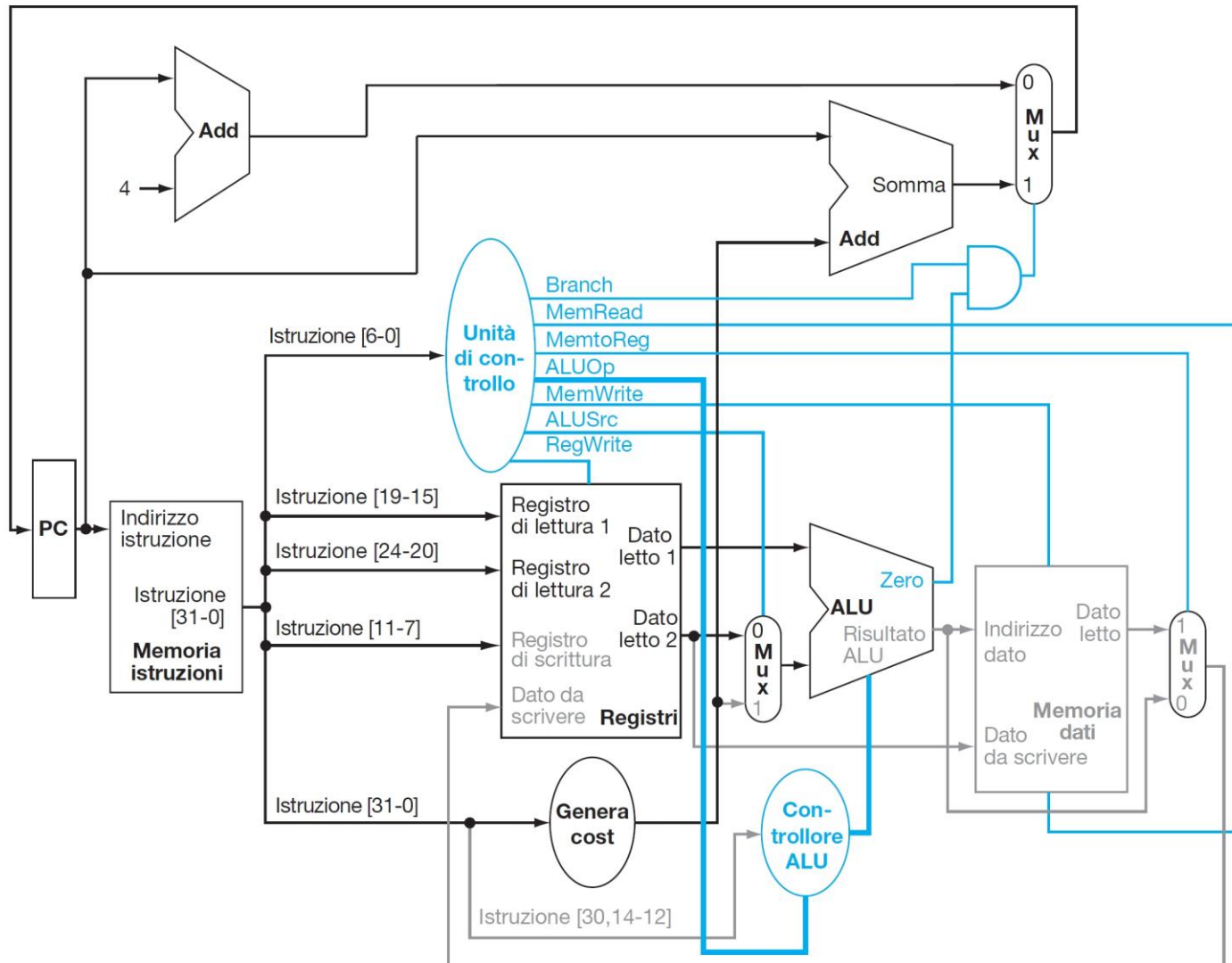
Esecuzione di un'istruzione di tipo R



Esecuzione di lw



Esecuzione di beq



Segnali di controllo

Nome del segnale	Effetto quando non asserito	Effetto quando asserito
RegWrite	Nulla	Il dato viene scritto nel register file nel registro individuato dal numero del registro di scrittura
ALUSrc	Il secondo operando della ALU proviene dalla seconda uscita del register file (Dato letto 2)	Il secondo operando della ALU proviene dall'estensione del segno dei 12 bit del campo immediato dell'istruzione
PCSrc	Nel PC viene scritta l'uscita del sommatore che calcola il valore di $PC + 4$	Nel PC viene scritta l'uscita del sommatore che calcola l'indirizzo di salto
MemRead	Nulla	Il dato della memoria nella posizione puntata dall'indirizzo viene inviato in uscita sulla linea "Dato letto"
MemWrite	Nulla	Il contenuto della memoria nella posizione puntata dall'indirizzo viene sostituito con il dato presente sulla linea "Dato scritto"
MemtoReg	Il dato inviato al register file per la scrittura proviene dalla ALU	Il dato inviato al register file per la scrittura proviene dalla Memoria Dati

I segnali da generare

L'ALU deve seguire 4 tipi di comportamento:

- Se l'istruzione è di **tipo R** eseguire l'operazione indicata dal campo **funz** dell'istruzione
- Se l'istruzione accede alla memoria (**lw, sw**) svolgere la **somma** che calcola l'indirizzo
- Se l'istruzione è un **beq** deve svolgere una **differenza**

Per codificare 3 comportamenti bastano 2 segnali dalla CU: **ALUOp1** ed **ALUOp0**

Quindi i segnali che la CU deve produrre per i diversi tipi di istruzione devono essere:

Istruzione	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp1	ALUOp0
Tipo R	0	0	1	0	0	0	1	0
lw	1	1	1	1	0	0	0	0
sw	1	X	0	0	1	0	0	0
beq	0	X	0	0	0	1	0	1

X indica 'don't care' (termini indifferenti)

Tempi di esecuzione

Conoscendo il tempo necessario a produrre i risultati delle diverse unità funzionali è possibile calcolare il tempo totale di ciascuna istruzione. Supponiamo che i tempi siano:

accesso alla memoria (dati o istruzione)	= 100 ns	(Fetch e MEM)
ALU e sommatori	= 150 ns	(EX e PC)
accesso ai registri (in lettura o scrittura)	= 50 ns	(ID e WB)
tutte le altre componenti	= 0 ns	

Allora i tempi di esecuzione delle istruzioni saranno

Tipo di istruzione	Lettura dell'istruzione	Lettura dei registri	Operazione con la ALU	Accesso ai dati in memoria	Scrittura del register file	Tempo totale
Load word (lw)	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
Store word (sw)	200 ps	100 ps	200 ps	200 ps		700 ps
Formato R (add, sub e or)	200 ps	100 ps	200 ps		100 ps	600 ps
Salto condizionato (beq)	200 ps	100 ps	200 ps			500 ps

NOTA: le due operazioni di somma per calcolare PC +4 (150ns) e salti condizionati (altri 150ns) sono svolte in parallelo al Fetch, Decode ed Execution e non allungano i tempi